In-situ Neural Data Compression for Gyrokinetic Exascale Simulations

A. de Vora[1], P. Krah[1], V. Grandgirard[1], A. Bermeo Marinelli[2], J. Bigot[3], E. Bourne[4], P. Donnel[1], I. Lucas[3]
P. Houngbasse[1], T. Moreau[5], K. Obrejan[1], T. Padioleau[3], B. Martin[3], B. Raffin[2], H. Verninas[5], Y. Wang[3]

[1]*CEA, IRFM, F-13108 Saint-Paul-lez-Durance, France,* [2]*Inria, CNRS, Université Grenoble Alpes, 38100, Grenoble, France,*
[3]*Université Paris-Saclay, UVSQ, CNRS, CEA, Maison de la Simulation, 91191, Gif-sur-Yvette, France,*
[4]*SCITAS, EPFL, CH-1015 Lausanne, Switzerland,* [5]*Université Paris-Saclay, Inria, CEA, 91120, Palaiseau, France*

3-7. August 2026, ICDDPS-7, Kiel, Germany

Motivation & objectives

This poster introduces an **in-situ data compression** strategy *designed for* gyrokinetic simulations [1] inside **Gyselalib++** [2], with a spotlight on:

- **Preserving key physics**: impact of the compression on conserved quantities
- **Scaling to the exascale era**: performance on massively parallel system

Key challenge

Efficient compression of distribution functions, while preserving physical structure.

Physical model

Vlasov Poisson 2D+2V

$$\partial_t f + \boldsymbol{v}\cdot\nabla_{\boldsymbol{x}} f + \boldsymbol{E}\cdot\nabla_{\boldsymbol{v}} f = 0, \quad \nabla\cdot\boldsymbol{E} = 1 - \int f \,\mathrm{d}\boldsymbol{v} \tag{1}$$

with initial condition (*Two Stream Instability*):

$$f_0(\boldsymbol{x},\boldsymbol{v}) = \frac{1}{2\pi}\left(e^{-\frac{1}{2}\|\boldsymbol{v}-\boldsymbol{v}_0\|^2} + e^{-\frac{1}{2}\|\boldsymbol{v}+\boldsymbol{v}_0\|^2}\right)(1+\alpha\cos(\boldsymbol{k}\cdot\boldsymbol{x})) \tag{2}$$

We measure *mass* $\mathscr{M}(t)$, *momentum* $\mathscr{P}(t)$, and *total energy* $\mathscr{E}(t) = \mathscr{E}_{\mathrm{kin}}(t) + \mathscr{E}_{\mathrm{pot}}(t)$, where

$$\mathscr{E}_{\mathrm{kin}}(t) = \frac{1}{2}\iint f\|\boldsymbol{v}\|^2 \,\mathrm{d}\boldsymbol{x}\,\mathrm{d}\boldsymbol{v}, \qquad \mathscr{E}_{\mathrm{pot}}(t) = \frac{1}{2}\int_{\Omega_x} \|\boldsymbol{E}(\boldsymbol{x},t)\|^2 \,\mathrm{d}\boldsymbol{x} \tag{3}$$

Compression: POD

Proper Orthogonal Decomposition (POD): truncated SVD with $r = 2, 8, 32$ modes,

$$f(\boldsymbol{x},\boldsymbol{v},t^k) \approx \tilde{f}^{\mathrm{POD}} = \sum_{l=1}^{r} \sigma_l X_l(\boldsymbol{x}) V_l(\boldsymbol{v}), \qquad \mathscr{E}^{\mathrm{POD}} = \sum_{k=r+1} \sigma_k$$

Implicit Neural Representations (INR)

At each simulation checkpoint t^k, we minimize the data loss over the N grid points:

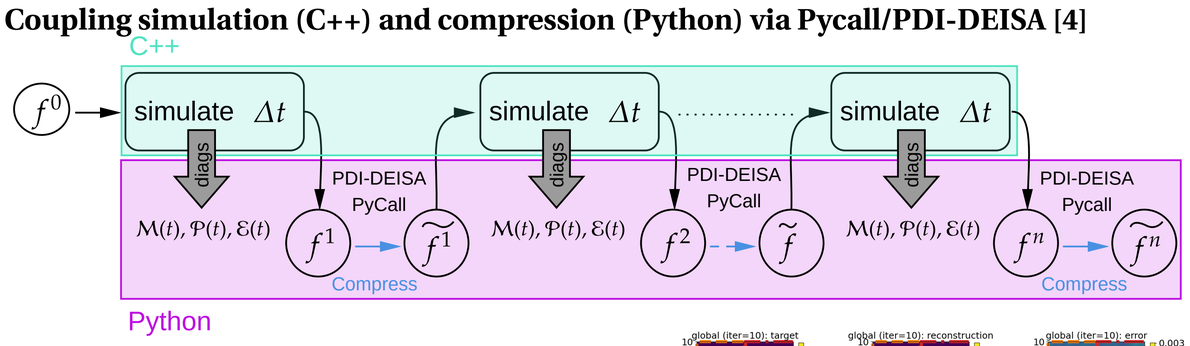
$$(\boldsymbol{x},\boldsymbol{v},t) \mapsto \left(\bigcirc\right) = \tilde{f}_\theta(\boldsymbol{x},\boldsymbol{v},t) \qquad \mathscr{L}^k_{\mathrm{data}} = \tfrac{1}{N}\textstyle\sum_{i,j}\left|f(x_i,v_j,t^k) - f_\theta(x_i,v_j,t^k)\right|^2$$

Best Architectures: SIREN MLP with 5 hidden layers and 128 neurons per layer
Two-Phase optimization loop (ADAM/L-BGFS) with **curriculum learning** using `scimba-jax` [3]:

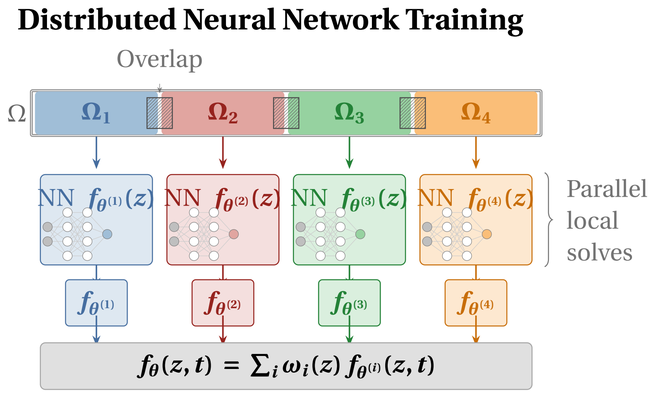
- **Cold start** (first checkpoint): `ADAM` (3000 iterations) + `L-BFGS` (100 iterations)
- **Warm start** (subsequent checkpoints): `ADAM` (300 iterations) + `L-BFGS` (10 iterations)
 The network at t^k is initialized with the optimized weights θ^* from t^{k-1}

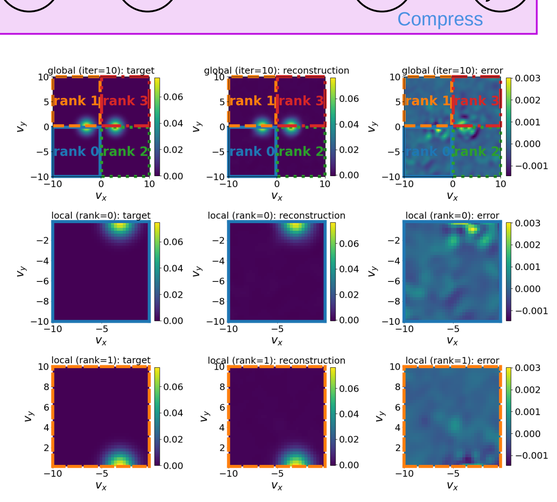
In-situ distributed data workflow

Coupling simulation (C++) and compression (Python) via Pycall/PDI-DEISA [4]



Distributed Neural Network Training



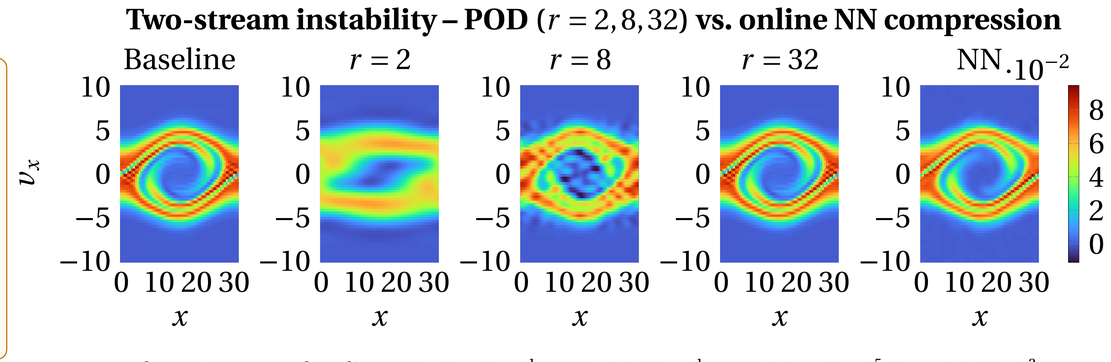


Results

Baseline Computation

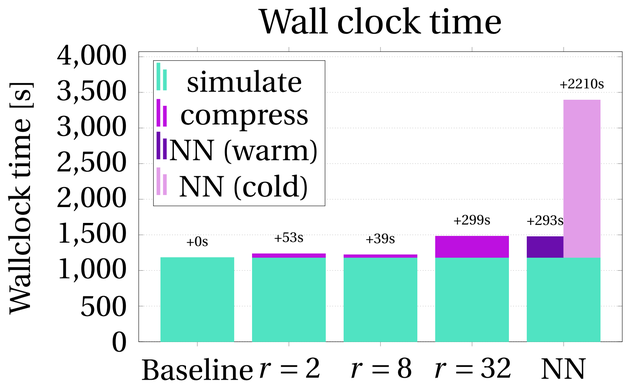
- Semi-Lagrangian Spline Solver (Gyselalib++ [2])
- $\Delta t = 0.1$
- checkpoint every 10 iterations
- mesh $64 \times 64 \times 65 \times 65$
- pertubation $\alpha = 0.05$
- pertubation mode $\boldsymbol{k}$ with $k_x = k_y = 0.2$

Two-stream instability – POD ($r = 2, 8, 32$) vs. online NN compression

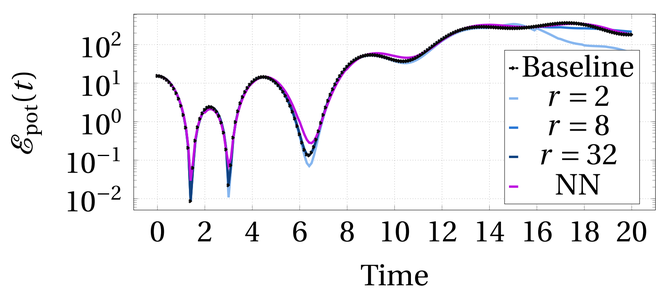


relative L_2 error vs. baseline: $r = 2$: 4.5×10^{-1} $r = 8$: 2.3×10^{-1} $r = 32$: 3.5×10^{-5} NN: 1.5×10^{-2}

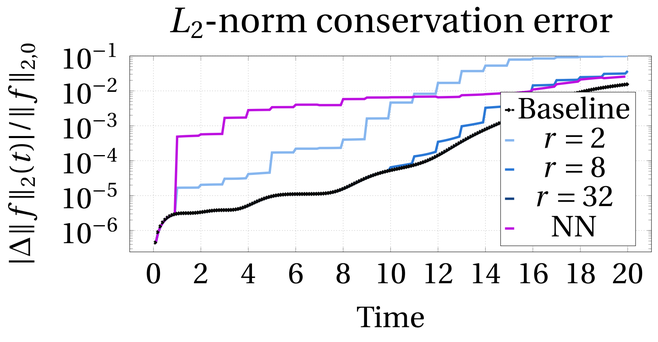
Wall clock time



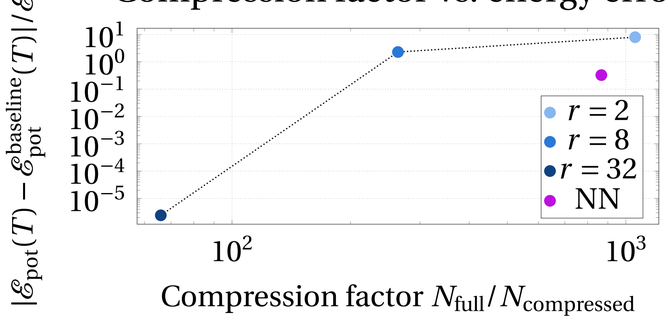
Electric potential energy



L_2-norm conservation error



Compression factor vs. energy error



Conclusions

- **Versatile in-situ framework**: a lightweight PDI-DEISA/Pycall [4] bridge exposes simulation data to Python, running compression and training on dedicated GPU/CPU nodes decoupled from the simulation – supporting both classical (POD) and advanced neural-network compression via `scimba-jax` [3], and portable to other simulation codes
- **Online POD**: $r = 32$ modes track the baseline within 10^{-5} relative error in $\mathscr{E}_{\mathrm{pot}}$ and phase space, at comparable CPU/GPU cost – but with a low compression ratio
- **Online NN**: achieves a high compression ratio with small relative error and supports distributed training that adapts to the code's parallelization strategy

Outlook

- Speed up NN training: better optimizers & stopping criteria,
- Test on 5D gyrokinetic simulations in the turbulent regime
- Surrogate-model pipeline for optimization & parameter scans
- Include distributed PINN training

Contact: `ariel.devora/philipp.krah@cea.fr`

Literature

[1] V. Grandgirard et al. "A 5D gyrokinetic full-f global semi-Lagrangian code for flux-driven ion turbulence simulations". In: *Comput. Phys. Commun.* 207 (2016), pp. 35–68. ISSN: 0010-4655. DOI: `https://doi.org/10.1016/j.cpc.2016.05.007`. URL: `https://www.sciencedirect.com/science/article/pii/S0010465516301230`.

[2] Emily Bourne et al. "Gyselalib++: A portable c++ library for semi-lagrangian kinetic and gyrokinetic simulations". In: *J. Open Source Softw.* 10.113 (2025), p. 8582.

[3] E.Franck, R. Imbach, V. Michel-Dansac. *Scimba Python library*. July 2026. URL: `https://gitlab.com/scimba/scimba/`.

[4] Amal Gueroudji, Julien Bigot, and Bruno Raffin. "DEISA: dask-enabled in situ analytics". In: *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE. 2021, pp. 11–20.

LaTeX TikZposter